

Lecture 4

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- expose errors of a different form: rounding error
- highlight IEEE-754 standard
- loss of significance
- cancellation

Floating Points

Normalized Floating-Point Representation

Real numbers are stored as

$$x = \pm(d_0.d_1d_2d_3 \dots d_m)_\beta \times \beta^e$$

- $d_1d_2d_3 \dots d_m$ is the mantissa, e is the exponent
- e is negative, positive or zero
- in normalized form we assume $d_0 = 1$ and do not store the bit

Floating Point

Example

In base 10

- 1000.12345 can be written as

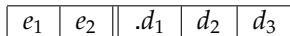
$$(0.100012345)_{10} \times 10^4$$

- 0.000812345 can be written as

$$(0.812345)_{10} \times 10^{-3}$$

Floating Point

Suppose we have only 3 bits for a mantissa and a 2 bit exponent stored like



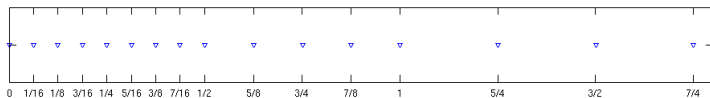
where the fraction is in the form $q \times 2^m$. Also assume $m = c - 1$ with q and c represented by bits d_i and e_i respectively. Assume $e_1e_2 = 11$ is reserved for the special case of $+\infty$. With $m = c - 1$ we have $m = 0 - 1, 1 - 1, 2 - 1 = -1, 0, 1$. All possible combinations give:

$$000_2 = 0$$

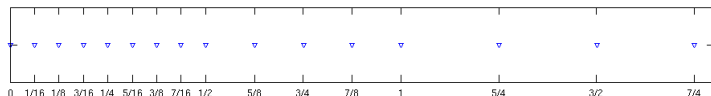
$$\dots \times 2^{-1,0,1}$$

$$111_2 = 7$$

So we get $0, \frac{1}{16}, \frac{2}{16}, \dots, \frac{7}{16}, 0, \frac{1}{4}, \frac{2}{4}, \dots, \frac{7}{4}$, and $0, \frac{1}{8}, \frac{2}{8}, \dots, \frac{7}{8}$. On the real line:



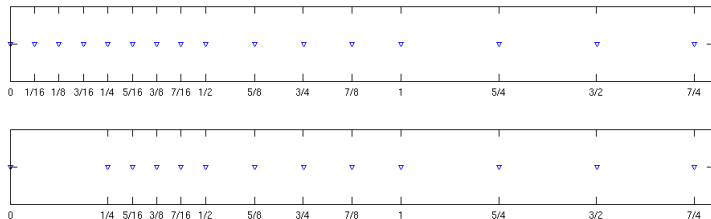
Overflow, Underflow



- computations too close to zero may result in *underflow*
- computations too large may result in *overflow*
- overflow error is considered more severe
- underflow can just fall back to 0

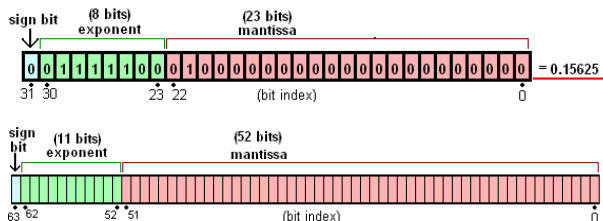
Normalizing

If we use the normalized form in our 4-bit case, we lose $0.001_2 \times 2^{-1,0,1}$ along with other. So we cannot represent $\frac{1}{16}$, $\frac{1}{8}$, and $\frac{3}{16}$.



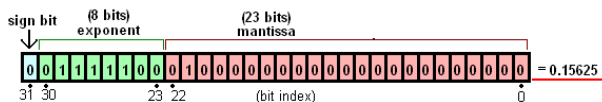
IEEE Floating Point (v. 754)

- How much storage do we actually use in practice?
- 32-bit word lengths are typical
- IEEE Single-precision floating-point numbers use 32 bits
- IEEE Double-precision floating-point numbers use 64 bits
- Goal: use the 32-bits to best represent the normalized floating point number



IEEE Single Precision (Marc-32)

$$x = \pm q \times 2^m$$



Notes:

- 1-bit sign
- 8-bit exponent $|m|$
- 23-bit mantissa q
- The leading one in the mantissa q does not need to be represented:
 $b_1 = 1$ is hidden bit
- IEEE 754: put x in $1.f$ normalized form
- $0 < m + 127 = c < 255$
- Largest exponent = 127, Smallest exponent = -126
- Special cases: $c = 0, 255$
- use 1-bit sign for negatives (1) and positives (0)

IEEE Single Precision

$$x = \pm q \times 2^m$$

Process for $x = -52.125$:

- 1 Convert both integer and fractional to binary:

$$x = -(110100.00100000000)_2$$

- 2 Convert to 1.f form: $x = \underbrace{-}_1 (1.\underbrace{101\,000\,010\,000\,\dots\,0}_{23})_2 \times 2^5$

- 3 Convert exponent $5 = c - 127 \Rightarrow c = 132 \Rightarrow c = (\underbrace{10\,000\,100}_8)_2$

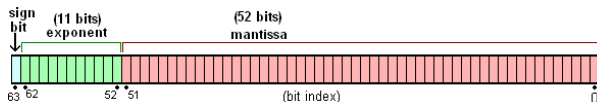
$$\underbrace{1}_1 \underbrace{10\,000\,100}_8 \underbrace{101\,000\,010\,000\,\dots\,0}_{23}$$

IEEE Single Precision

Special Cases:

- denormalized/subnormal numbers: use 1 extra bit in the significant: exponent is now -126 (less precision, more range), indicated by 00000000_2 in the exponent field
- two zeros: $+0$ and -0 (0 mantissa, 0 exponent)
- two ∞ 's: $+\infty$ and $-\infty$
- ∞ (0 mantissa, 11111111_2 exponent)
- NaN (any mantissa, 11111111_2 exponent)
- see appendix C.1 in NMC 6th ed.

IEEE Double Precision



- 1-bit sign
- 11-bit exponent
- 52-bit mantissa
- single-precision: about 6 decimal digits of precision
- double-precision: about 15 decimal digits of precision
- $m = c - 1023$

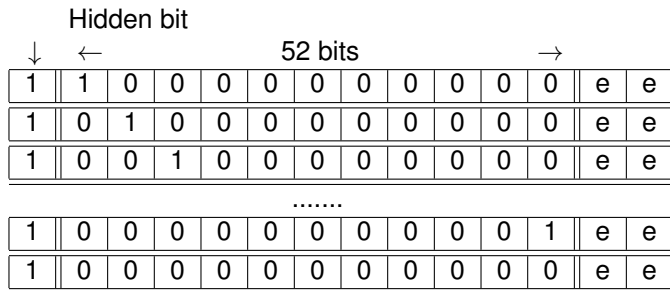
Precision vs. Range

type	range	approx range
single	$-3.40 \times 10^{38} \leq x \leq -1.18 \times 10^{-38}$ 0 $1.18 \times 10^{-38} \leq x \leq 3.40 \times 10^{38}$	$2^{-126} \rightarrow 2^{128}$
double	$-1.80 \times 10^{318} \leq x \leq -2.23 \times 10^{-308}$ 0 $2.23 \times 10^{-308} \leq x \leq 1.80 \times 10^{308}$	$2^{-1022} \rightarrow 2^{1024}$

Listing 1: Matlab Tools

```
1 >> realmax
2 ans = 1.7977e+308
3
4 >> realmin
5 ans = 2.2251e-308
```

Take $x = 1.0$ and add $1/2, 1/4, \dots, 2^{-i}$:



- Ooops!
- use $fl(x)$ to represent the floating point machine number for the real number x
- $fl(1 + 2^{-52}) \neq 1$, but $fl(1 + 2^{-53}) = 1$

ϵ_m

Machine Epsilon

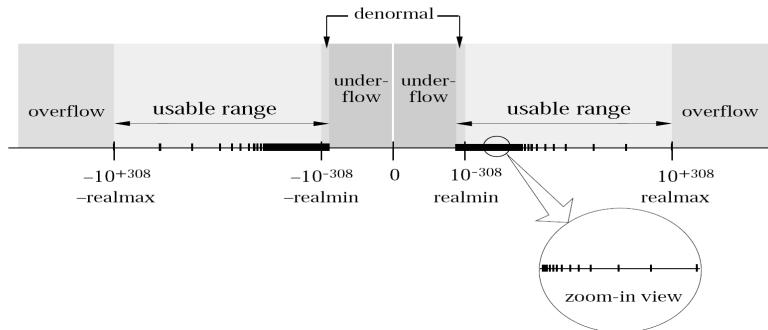
The machine epsilon ϵ_m is the smallest number such that

$$fl(1 + \epsilon_m) \neq 1$$

- The double precision machine epsilon is about 2^{-52} .
- The single precision machine epsilon is about 2^{-23} .

```
1 >> eps
2 ans = 2.2204e-16
3
4 >> eps('single')
5 ans = 1.1921e-07
```

Floating Point Number Line



Floating Point Errors

- Not all reals can be exactly represented as a machine floating point number. Then what?
- Round-off error
- IEEE options:
 - Round to next nearest FP (preferred), Round to 0, Round up, and Round down

Let x_+ and x_- be the two floating point machine numbers closest to x

- round to nearest: $round(x) = x_-$ or x_+ , whichever is closest
- round toward 0: $round(x) = x_-$ or x_+ , whichever is between 0 and x
- round toward $-\infty$ (down): $round(x) = x_-$
- round toward $+\infty$ (up): $round(x) = x_+$

Floating Point Errors

How big is this error? Suppose (x is closer to x_-)

$$x = (0.1b_2b_3 \dots b_{24}b_{25}b_{26})_2 \times 2^m$$

$$x_- = (0.1b_2b_3 \dots b_{24})_2 \times 2^m$$

$$x_+ = ((0.1b_2b_3 \dots b_{24})_2 + 2^{-24}) \times 2^m$$

$$|x - x_-| \leq \frac{|x_+ - x_-|}{2} = 2^{m-25}$$

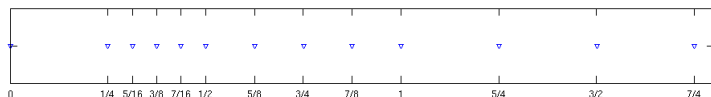
$$\left| \frac{x - x_-}{x} \right| \leq \frac{2^{m-25}}{1/2 \times 2^m} \leq 2^{-24} = \epsilon_m/2$$

Floating Point Arithmetic

- Problem: The set of representable machine numbers is FINITE.
- So not all math operations are well defined!
- Basic algebra breaks down in floating point arithmetic

Example

$$a + (b + c) \neq (a + b) + c$$



Floating Point Arithmetic

Rule 1.

$$fl(x) = x(1 + \epsilon), \quad \text{where} \quad |\epsilon| \leq \epsilon_m$$

Rule 2.

For all operations $\odot$ (one of $+$, $-$, $*$, $/$)

$$fl(x \odot y) = (x \odot y)(1 + \epsilon_{\odot}), \quad \text{where} \quad |\epsilon_{\odot}| \leq \epsilon_m$$

Rule 3.

For $+$, $*$ operations

$$fl(a \odot b) = fl(b \odot a)$$

There were many discussions on what conditions/ rules should be satisfied by floating point arithmetic. The IEEE standard is a set of standards adopted by many CPU manufacturers.

Floating Point Arithmetic

Consider the sum of 3 numbers: $y = a + b + c$.

Done as $fl(fl(a + b) + c)$

$$\begin{aligned}\eta &= fl(a + b) = (a + b)(1 + \epsilon_1) \\ y_1 &= fl(\eta + c) = (\eta + c)(1 + \epsilon_2) \\ &= [(a + b)(1 + \epsilon_1) + c](1 + \epsilon_2) \\ &= [(a + b + c) + (a + b)\epsilon_1](1 + \epsilon_2) \\ &= (a + b + c) \left[1 + \frac{a + b}{a + b + c} \epsilon_1 (1 + \epsilon_2) + \epsilon_2 \right]\end{aligned}$$

So disregarding the high order term $\epsilon_1 \epsilon_2$

$$fl(fl(a + b) + c) = (a + b + c)(1 + \epsilon_3) \quad \text{with} \quad \epsilon_3 \approx \frac{a + b}{a + b + c} \epsilon_1 + \epsilon_2$$

Floating Point Arithmetic

If we redid the computation as $y_2 = fl(a + fl(b + c))$ we would find

$$fl(a + fl(b + c)) = (a + b + c)(1 + \epsilon_4) \quad \text{with} \quad \epsilon_4 \approx \frac{b + c}{a + b + c} \epsilon_1 + \epsilon_2$$

Main conclusion:

The first error is amplified by the factor $(a + b)/y$ in the first case and $(b + c)/y$ in the second case.

In order to sum n numbers more accurately, it is better to start with the small numbers first. [However, sorting before adding is usually not worth the cost!]

Floating Point Arithmetic

Roundoff errors and floating-point arithmetic

One of the most serious problems in floating point arithmetic is that of cancellation. If two large and close-by numbers are subtracted the result (a small number) carries very few accurate digits (why?). This is fine if the result is not reused. If the result is part of another calculation, then there may be a serious problem

Example

Roots of the equation

$$x^2 + 2px - q = 0$$

Assume we want the root with smallest absolute value:

$$y = -p + \sqrt{p^2 + q} = \frac{q}{p + \sqrt{p^2 + q}}$$

Loss of Significance

Listing 2: alg 1

```
1  $s := p^2$   
2  $t := s + q$   
3  $u := \sqrt{t}$   
4  $y := -p + u$ 
```

Listing 3: alg 2

```
1  $s := p^2$   
2  $t := s + q$   
3  $u := \sqrt{t}$   
4  $v := p + u$   
5  $y := q/v$ 
```

Step 4 of Algorithm 1 can have severe cancellation when $p \gg q$ and when p is positive.

```

1 >> p = 10000;
2 >> q = 1;
3 >> y = -p+sqrt(p^2 + q)
4 y = 5.0000000055588316e-05
5
6 >> y2 = q/(p+sqrt(p^2+q))
7 y2 = 4.9999999875000000e-05
8
9 >> x = y;
10 >> x^2 + 2 * p * x - q
11 ans = 1.361766321927860e-08
12
13 >> x = y2;
14 >> x^2 + 2 * p * x - q
15 ans = -1.110223024625157e-16

```

Consider now the case when

$$p = -(1 + \epsilon/2) \quad \text{and} \quad q = -(1 + \epsilon)$$

Exact Roots? Take $\epsilon = 1.E - 08$ and use matlab:

```

1 x = - p - sqrt(p^2 + q)    --->    1.000000000500000
2 y = - p + sqrt(p^2 + q)    --->    1.000000000500000

```

Catastrophic Cancellation

Adding $c = a + b$ will result in a large error if

- $a \gg b$
- $a \ll b$

Let

$$a = x.xxx \cdots \times 10^0$$

$$b = y.yyy \cdots \times 10^{-8}$$

Then

$$\begin{array}{r} \text{finite precision} \\ \overbrace{x.xxx \ xxxx \ xxxx \ xxxx} \\ + \quad 0.000 \ 0000 \ yyy \ yyy \ yyy \ yyy \\ \hline = \quad x.xxx \ xxxx \ zzzz \ zzzz \quad \underbrace{yyyy \ yyy \ yyy \ yyy}_{\text{lost precision}} \end{array}$$

Catastrophic Cancellation

Subtracting $c = a - b$ will result in large error if $a \approx b$. For example

$$ax.xxxx\ xxxx\ xxx1 \overbrace{ssss\ \dots}^{\text{lost}}$$

$$bx.xxxx\ xxxx\ xxx0 \overbrace{tttt\ \dots}^{\text{lost}}$$

Then

$$\begin{array}{r} \text{finite precision} \\ \overbrace{x.xxx\ xxxx\ xxx1} \\ + \quad \overbrace{x.xxx\ xxxx\ xxx0} \\ \hline = \quad 0.000\ 0000\ 0001 \quad \underbrace{????\ ????}_{\text{lost precision}} \end{array}$$

Summary

- addition: $c = a + b$ if $a \gg b$ or $a \ll b$
- subtraction: $c = a - b$ if $a \approx b$
- catastrophic: caused by a single operation, not by an accumulation of errors
- can often be fixed by mathematical rearrangement

Loss of Significance

Example

$x = 0.3721448693$ and $y = 0.3720214371$. What is the relative error in $x - y$ in a computer with 5 decimal digits of accuracy?

$$\frac{|x - y - (\bar{x} - \bar{y})|}{|x - y|} = \frac{|0.3721448693 - 0.3720214371 - 0.37214 + 0.37202|}{|0.3721448693 - 0.3720214371|}$$
$$\approx 3 \times 10^{-2}$$

Loss of Significance

Loss of Precision Theorem

Let x and y be (normalized) floating point machine numbers with $x > y > 0$.

If $2^{-p} \leq 1 - \frac{y}{x} \leq 2^{-q}$ for positive integers p and q , the significant binary digits lost in calculating $x - y$ is between q and p .

Loss of Significance

Example

Consider $x = 37.593621$ and $y = 37.584216$.

$$2^{-11} < 1 - \frac{y}{x} = 0.0002501754 < 2^{-12}$$

So we lose 11 or 12 bits in the computation of $x - y$. yikes!

Example

Back to the other example (5 digits): $x = 0.37214$ and $y = 0.37202$.

$$10^{-4} < 1 - \frac{y}{x} = 0.00032 < 10^{-5}$$

So we lose 4 or 5 bits in the computation of $x - y$. Here, $x - y = 0.00012$ which has only 1 significant digit that we can be sure about

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Problem at $x \approx 0$.

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Problem at $x \approx 0$.

One type of fix:

$$\begin{aligned} f(x) &= \left(\sqrt{x^2 + 1} - 1 \right) \left(\frac{\sqrt{x^2 + 1} + 1}{\sqrt{x^2 + 1} + 1} \right) \\ &= \frac{x^2}{\sqrt{x^2 + 1} + 1} \end{aligned}$$

no subtraction!